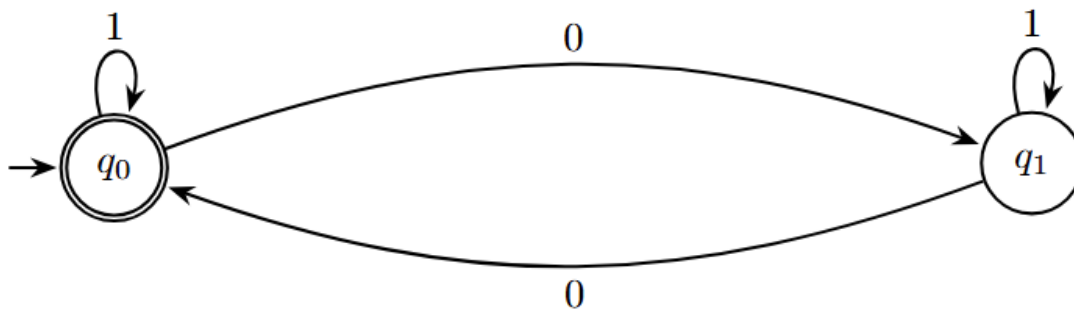


Interactive Environment for Designing and Simulating Deterministic Finite Automata

Example



The figure shows a simple automaton accepting the language L over the alphabet $\Sigma = 0, 1$, consisting of binary strings with an even number of zeros. The automaton will accept the word 100, while it will reject the word 10100.

Application Description

The application consists of two parts:

1. **Lab:** An automaton editor, which allows for creating an automaton as well as exporting/importing it from a file.
2. **Home:** A runtime environment, which allows importing an automaton from a file and simulating calculations on a given input word.

The subject of the laboratory task will be a **simplified automata editor**. The homework part of the project involves extending the editor's functionality and implementing the runtime environment.

Automaton Editor - laboratory task

The application consists of an interactive area used for drawing the automaton, as well as UI elements that allow modifying the automaton.

Editor functionalities:

- **Adding states:** (1.5p)
 - The "Add state" button adds a new state in the central part of the drawing area.
 - A state is represented as a circle with a label (see [Example](#)).
 - States are automatically numbered with consecutive natural numbers (e.g., $q_0, q_1, \dots, q_n$).
- **State activation:** (1p)
 - A single click on a state activates it.
 - The active element should be visually highlighted (e.g., by changing the border color).
 - Clicking on the drawing area where there is no state removes the highlight from the previously active state.
- **Changing state position:** (2p)
 - Pressing the right mouse button on a state, holding it, and moving the mouse changes the state's position.
 - Dropping the state occurs upon releasing the button.
- **Editing a state:** (2p)
 - Clicking the right mouse button on a state expands a context menu with the options:
 - "Mark as accepting"
 - "Mark as initial"
 - "Delete state"
 - An accepting state stands out from the other states (an example implementation is a double circle, see [Example](#)).
 - An initial state stands out from the other states.
 - Every valid automaton should have **exactly one** initial state (by default, it can be the first created state).
 - Marking a different state as initial automatically removes this designation from the previous initial state.
- **Adding a transition:** (1.5p)
 - For the activated state, a list of all states appears.
 - Each element on the list is a checkbox; checking it adds a new edge. Unchecking it removes the edge.
 - The checkbox for the same vertex is disabled.
 - At this stage, the "Start state" must be different from the "End state" (checking the Checkbox with the same state is disabled).

Hints

- Use the classes implemented in Model.cs.
- Events: MouseLeftButtonDown, MouseRightButtonDown, MouseLeftButtonUp, MouseMove.
- Canvas: [ItemsControl](#), [ItemsPanelTemplate](#), [Canvas](#).

- State rendering: `ItemTemplate`, `DataTemplate`, `DataTemplate.Triggers`.
- Shapes: `Line`, `Ellipse`.
- Context menu: `ContextMenu`, `MenuItem`.
- Controls: `ListBox`, `CheckBox`.